



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO2 ASSESSMENT TOOL 2
Architecture Design Assignment

SIMATS ENGINEERING
Department of Computer Science and Engineering

**Problem Statement 25: AI-Based Smart Microgrid Energy
Management and Load Balancing Platform**

NAME: MURALI KRISHNAN N

REG NO: 192524090

CODE: CSA1017

1. Analysis of System Requirements

1.1 System Objectives

The proposed platform is an AI-based Smart Microgrid Energy Management and Load Balancing system intended for campuses, industrial parks, and residential communities operating a microgrid. The system objectives, derived directly from the problem statement, are:

- Continuously monitor electricity generation (renewable and conventional) and consumption across the microgrid.
- Forecast short-term and medium-term electricity demand using AI/ML models.
- Optimize charging and discharging of battery energy storage systems (BESS).
- Automatically balance electrical loads across feeders/circuits in real time.
- Integrate intermittent renewable sources (solar, wind) into dispatch decisions.
- Generate energy performance and cost-saving reports for stakeholders.
- Reduce operational expenses and improve overall microgrid reliability and sustainability.

1.2 Functional Requirements (FR)

ID	Functional Requirement
FR1	Acquire real-time telemetry from smart meters, battery management systems (BMS), and renewable-source sensors via IoT gateways.
FR2	Forecast electricity demand and renewable generation using AI/ML time-series models (e.g., LSTM, gradient boosting).
FR3	Compute optimal battery charge/discharge schedules based on forecasted demand, tariff signals, and battery state-of-health.
FR4	Detect load imbalance and automatically issue load-shedding / load-shifting / re-routing commands to controllers.
FR5	Provide a real-time operational dashboard showing generation, consumption, storage level, and alerts.
FR6	Generate periodic (daily/weekly/monthly) energy performance and cost-savings reports.
FR7	Send alerts/notifications (SMS, email, push) for anomalies, faults, or critical storage thresholds.
FR8	Allow authorized operators to override automated decisions manually through the dashboard.
FR9	Integrate with external utility/grid systems for import/export of power and demand-response signals.

1.3 Non-Functional Requirements (NFR) and Quality Attributes

Because the platform controls physical energy infrastructure and must operate continuously under variable renewable input, the following non-functional requirements strongly influence the architectural style chosen:

Quality Attribute	Requirement / Influence on Architecture
Scalability	Must scale horizontally as more meters, sensors, buildings, and microgrid sites are added, and must handle bursty telemetry (thousands of readings/second) without redesign.
Performance	Load-balancing and control decisions must be computed and dispatched within seconds; dashboards must refresh near real time (sub-5-second latency).
Reliability	The system must keep operating (in a degraded/local mode) even if the cloud AI services are temporarily unreachable, since energy control is safety-critical.
Availability	Target availability of 99.9% or higher; no single point of failure in telemetry ingestion or control paths.
Security	Sensor data, control commands, and dashboards must be protected against spoofing, replay, and unauthorized access (IoT devices are a common attack surface).
Maintainability	AI forecasting models must be retrained/updated and new device types onboarded without redeploying the entire system.
Interoperability	Must integrate heterogeneous protocols (MQTT, Modbus, OPC-UA) and external systems (utility grid, weather APIs).

These attributes indicate a system that is data-intensive, real-time, loosely coupled, independently scalable per function, and resilient to partial failure — pointing toward a hybrid, event-driven, service-oriented style rather than a single monolithic layered application.

2. Selection of Suitable Software Architecture

2.1 Selected Architectural Style

Selected Style: A Hybrid Architecture combining Layered Architecture, Event-Driven Architecture (EDA), and Microservices.

Layering is used for overall organization (Field/IoT → Edge/Gateway → Event Bus → Application Services → Data → API/Presentation → Client), event-driven messaging (via a Kafka/MQTT-based broker) decouples fast-arriving IoT telemetry from processing services, and each core capability (forecasting, battery optimization, load balancing, monitoring, reporting) is implemented as an independently deployable microservice.

2.2 Justification for Selection

- Pure Layered Architecture alone is too rigid: AI forecasting and load-balancing must react to continuous, high-volume sensor streams, not simple synchronous request/response calls typical of strict layering.
- Pure Microservices without an event backbone would force services to poll each other constantly for telemetry, adding latency and coupling; an event bus lets thousands of readings/second be published once and consumed by multiple services (forecasting, monitoring, reporting) independently.
- Event-Driven Architecture (EDA) fits the asynchronous, continuous nature of energy telemetry and control commands, and naturally supports real-time alerting and reactive load-shedding.
- Microservices allow each AI/optimization capability (forecasting, battery optimization, load balancing) to be scaled, updated, and retrained independently — critical since AI models evolve faster than the rest of the system.
- A lightweight layered organization is still retained at the macro level (edge, integration, application, data, presentation) so the overall system remains understandable, testable, and maintainable rather than an unstructured mesh of services.

This hybrid combination directly satisfies the scalability, performance, reliability, and maintainability requirements identified in Section 1, while keeping the design comprehensible enough for a small operations/engineering team to maintain.

3. Software Architecture Diagram

The diagram below illustrates the complete structure of the proposed system, including all major layers, the event bus, AI-driven microservices, data stores, external systems, and client applications, along with the primary communication mechanisms (MQTT/HTTPS ingestion, Kafka event streaming, REST/GraphQL APIs).

AI-Based Smart Microgrid Energy Management & Load Balancing Platform

Hybrid Architecture: Layered + Event-Driven Microservices

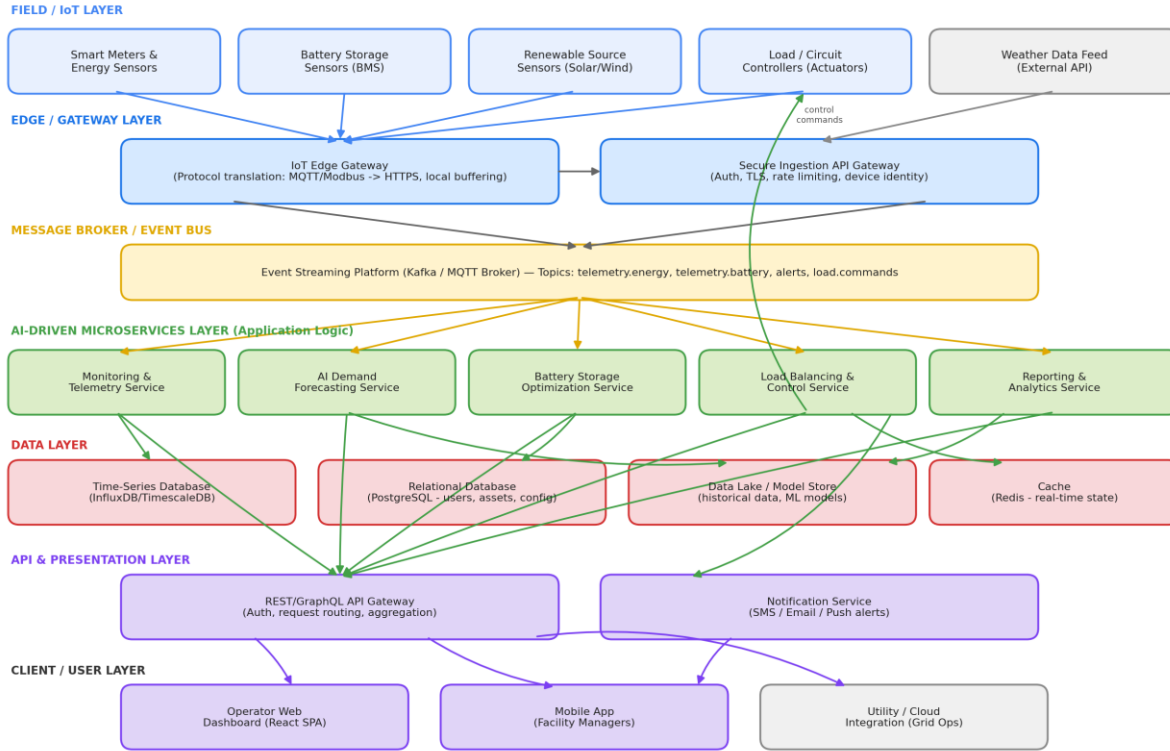


Figure 1: Hybrid Layered + Event-Driven Microservices Architecture for the Smart Microgrid Platform

4. Explanation of Components and Their Interactions

4.1 Component Responsibilities

Component	Purpose / Responsibility
Field/IoT Layer (Smart meters, BMS sensors, renewable sensors, load controllers)	Capture raw physical measurements (voltage, current, power, state-of-charge, irradiance/wind speed) and execute control actions (breaker/relay switching) issued by the Load Balancing Service.
IoT Edge Gateway	Translates device protocols (MQTT/Modbus/OPC-UA) into a common format, performs local buffering during connectivity loss, and forwards telemetry upstream — enabling limited offline operation.
Secure Ingestion API Gateway	Authenticates devices/services, enforces TLS and rate limiting, and validates incoming payloads before they reach the event bus.

Event Streaming Platform (Kafka/MQTT broker)	Acts as the central nervous system of the platform; decouples producers (sensors, gateways) from consumers (microservices) via topics such as telemetry.energy, telemetry.battery, alerts, and load.commands.
Monitoring & Telemetry Service	Continuously consumes live telemetry, computes real-time KPIs, and raises threshold-based alerts.
AI Demand Forecasting Service	Consumes historical and streaming data to forecast electricity demand and renewable generation using ML models retrieved from the Data Lake/Model Store.
Battery Storage Optimization Service	Uses forecasts and current state-of-charge to compute optimal charge/discharge schedules that minimize cost and extend battery life.
Load Balancing & Control Service	Combines forecasts and optimization output to detect imbalance and issue automated load-shifting/shedding commands back to field controllers via the event bus.
Reporting & Analytics Service	Aggregates historical data into periodic energy performance, cost-saving, and sustainability reports.
Data Layer (Time-series DB, Relational DB, Data Lake, Cache)	Persist telemetry (time-series DB), configuration/user data (relational DB), historical data and trained ML models (data lake), and fast-changing real-time state (cache).
API Gateway (REST/GraphQL) & Notification Service	Expose aggregated data securely to client applications and push alerts/notifications to operators.
Operator Dashboard, Mobile App, Utility/Cloud Integration	Provide human-facing visualization and control, and exchange import/export and demand-response signals with the external electricity grid.

4.2 Component Interaction and Workflow

The overall data and control flow proceeds as follows:

- 1. Sensors and meters continuously publish readings to the IoT Edge Gateway using lightweight protocols (MQTT/Modbus).
- 2. The Edge Gateway forwards validated telemetry through the Secure Ingestion API Gateway to the Event Streaming Platform.
- 3. The Monitoring, Forecasting, Optimization, and Reporting microservices each subscribe independently to the relevant topics, so a single telemetry event can trigger multiple parallel reactions without services calling each other directly.
- 4. The AI Demand Forecasting Service produces demand/generation forecasts, which are consumed by the Battery Optimization Service and the Load Balancing Service.

- 5. The Load Balancing & Control Service evaluates current load versus forecast and available storage, then publishes control commands back onto the event bus, which the Edge Gateway delivers to field actuators/controllers.
- 6. Processed and raw data are persisted into the appropriate data stores (time-series DB for telemetry, data lake for model training data, cache for live dashboard state).
- 7. The API Gateway aggregates data from microservices and data stores and exposes it to the Operator Dashboard and Mobile App; the Notification Service pushes alerts when thresholds are breached.
- 8. The Utility/Cloud Integration component exchanges demand-response and grid-import/export signals with the external utility, closing the loop with the wider power grid.

This asynchronous, publish–subscribe workflow ensures that a burst of sensor data, a slow AI computation, and a fast dashboard refresh can all proceed concurrently without blocking one another.

5. Discussion of Quality Attributes

Attribute	How the Architecture Addresses It
Scalability	Each microservice (forecasting, optimization, load balancing, monitoring, reporting) can be scaled independently and horizontally behind the event bus; Kafka partitions allow ingestion throughput to grow simply by adding partitions/consumers as more meters and sites are onboarded.
Security	The Secure Ingestion API Gateway enforces device authentication, TLS encryption, and rate limiting at the edge; the API Gateway enforces role-based access control for operators; sensitive control-command topics can be access-restricted separately from read-only telemetry topics.
Performance	The event bus enables near real-time (sub-second to few-second) propagation of telemetry to consumers; caching (Redis) serves live dashboard state without repeatedly querying the time-series database, keeping UI refresh latency low.
Reliability	Local buffering at the Edge Gateway allows continued data capture during connectivity loss; because services communicate via durable event topics rather than direct synchronous calls, a temporarily unavailable service (e.g., forecasting) does not block ingestion or basic monitoring/alerting.
Maintainability	Independently deployable microservices mean the AI forecasting model can be retrained and redeployed without touching the load-balancing or monitoring services; new device protocols can be added at the Edge Gateway without changing downstream services.
Availability	Removing tight coupling between services and using a replicated, distributed event broker avoids a single point of failure; critical control logic (Load Balancing Service) can be replicated across nodes for failover.

6. Justification of Architectural Decisions

6.1 Rationale

The hybrid Layered + Event-Driven + Microservices approach was chosen because no single classical style fully satisfies the microgrid's requirements in isolation. A strictly Layered Architecture would provide clarity but could not efficiently handle the continuous, high-frequency, asynchronous sensor streams and would create tight coupling between the forecasting, optimization, and control logic. A pure Microservices architecture without an event backbone would require excessive synchronous service-to-service calls for shared telemetry, increasing latency and reducing resilience. Combining the three styles allows the system to remain organized (layered view), reactive to continuous data (event-driven), and independently scalable per capability (microservices).

6.2 Trade-offs Considered

- **Complexity vs. Flexibility:** The hybrid approach introduces more operational complexity (multiple services, a message broker, several data stores) than a monolith, but this is justified by the independent scaling and update needs of the AI components.
- **Eventual Consistency vs. Real-Time Responsiveness:** Asynchronous messaging can introduce small propagation delays and eventual consistency between services, which was accepted because sub-second/near-real-time behaviour is sufficient for energy monitoring and control, unlike, for example, financial transaction systems.
- **Operational Overhead vs. Resilience:** Running and monitoring a distributed message broker and multiple services requires more DevOps effort than a single application, but was chosen to avoid a single point of failure in a safety-relevant control system.
- **Build vs. Reuse:** Using established open-source components (Kafka/MQTT, TimescaleDB/InfluxDB, PostgreSQL, Redis) rather than custom-built equivalents reduces development risk and speeds delivery, at the cost of dependency on external ecosystems.

6.3 Support for Future Enhancements, Integration, and Long-Term Maintainability

- New renewable sources, storage units, or microgrid sites can be onboarded by simply registering new devices at the Edge Gateway and publishing to existing event topics — no redesign of core services is required.
- Additional AI models (e.g., predictive maintenance for batteries, anomaly detection for cyberattacks) can be added as new microservices that subscribe to existing telemetry topics without disrupting current services.
- Integration with wider smart-grid or utility demand-response programs is straightforward through the existing API Gateway and Utility/Cloud Integration component.

- Because each microservice owns its own logic and can be independently versioned and redeployed, the platform supports continuous improvement (e.g., periodic retraining of forecasting models) with minimal downtime, supporting long-term maintainability.

6.4 Conclusion

The proposed Hybrid Layered + Event-Driven Microservices Architecture directly addresses the functional requirements (monitoring, forecasting, battery optimization, load balancing, reporting) and the non-functional requirements (scalability, performance, reliability, availability, security, maintainability) identified in the problem statement. It provides a resilient, extensible foundation capable of supporting the stated objective of improving energy reliability, reducing operational costs, and enabling sustainable, data-driven power management for the microgrid.